Actividad 2.5

Automatización del despliegue de contenedores con GitHub Actions y AWS

ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

Sigla/Asignatura	Semana Disponible EFT	Experiencia de Aprendizaje
ISY1101 – Introducción a Herramientas DevOps	**17**	**EA 2**
Tiempo	**Modalidad de Trabajo**	**Indicadores de Logro**
5 hrs	**Presencial**	**IL 2.5**

Resultados de aprendizaje

- **RA2** Configura contenedores de software utilizando herramientas de los distintos proveedores para encapsular el código, facilitando el manejo y uso de recursos optimizado.

Contenidos

- Al finalizar la sesión, los/las estudiantes serán capaces de configurar workflows en GitHub Actions, automatizar la construcción y publicación de imágenes Docker, desplegar aplicaciones en AWS EC2 y comprender cómo la CI/CD optimiza la entrega continua dentro de un flujo DevOps profesional.

Actividades

Actividad: Construye y despliegue pipeline CICD con GitHub Actions

En esta actividad, las y los estudiantes deberán **diseñar, configurar y ejecutar un pipeline de Integración y Entrega Continua (CI/CD)** utilizando **GitHub Actions**, automatizando el proceso de **build, publicación y despliegue** de una aplicación contenerizada por capas (frontend, backend y base de datos).

La actividad considera la **creación de un repositorio en GitHub**, la definición de **workflows CI/CD independientes por cada capa**, la **configuración de secretos y variables de entorno** para una integración segura con servicios de AWS, y la **ejecución del pipeline ante cambios en el código fuente**.

Como parte del proceso, el pipeline deberá **construir nuevas imágenes Docker**, **publicarlas en Amazon ECR** y **desplegarlas automáticamente en instancias EC2** mediante **AWS Systems Manager (SSM)**, reemplazando los contenedores existentes sin intervención manual. Finalmente, las y los estudiantes deberán **validar el despliegue**, comprobando que la aplicación se actualiza correctamente tras un cambio en el repositorio.

Esta actividad permite comprender de forma práctica cómo los pipelines CI/CD facilitan la **automatización, trazabilidad y repetibilidad** de los despliegues, consolidando el uso de **GitHub Actions, Docker, ECR y EC2** dentro de un flujo DevOps real.

Actividad guiada en clase

1. Los estudiantes deben:
2. Crear un repositorio GitHub para la app "tienda perritos".
3. Agregar workflows CI/CD en Actions por cada capa.
4. Configurar secretos y variables de seguridad
5. Ejecutar el pipeline.
6. Validar que el pipeline descargue la nueva imagen y reemplace el contenedor.

PASO1: Preparar el repositorio en GitHub

- Subir el proyecto con sus carpetas: frontend/, backend/, db/ a través de comandos GIT.
- Cada carpeta tiene su propio Dockerfile y cada capa tiene su propio Archivo workflow para ser copiado su contenido **(crear workflows en PASO 2):** cicd-tienda-db.yml - cicd-tienda-backend.yml - cicd-tienda-frontend.yml.

PASO 2: Configurar secretos en GitHub (Settings → Secrets and variables → Actions)

- **Credenciales AWS:** AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_SESSION_TOKEN, AWS_REGION.
- **Repositorios ECR:** ECR_REPO_URL_DB, ECR_REPO_URL_BACKEND, ECR_REPO_URL_FRONTEND, ECR_REGISTRY
- **Instancias EC2:** EC2_DB_INSTANCE_ID, EC2_BACKEND_INSTANCE_ID, EC2_FRONTEND_INSTANCE_ID.
- **Crear Workflow en:** Actions → New Workflow → set up a workflow yourself → nombre archivo workflow y su contenido → Commit changes

Copiar códigos para workflows (cicd-tienda-db.yml - cicd-tienda-backend.yml - cicd-tienda-frontend.yml)

PASO 3: Entender la lógica de cada pipeline (por capa)

- **Trigger:** se ejecuta cuando hay cambios en su carpeta (db/**, backend/**, frontend/**).
- **Etapas comunes:**
  - checkout del código.
  - configure-aws-credentials con los secrets.
  - amazon-ecr-login (login en ECR en el runner).
  - docker build de la imagen de la capa (DB / Backend / Frontend).
  - docker tag + docker push a su repo de **Amazon ECR**.

PASO 4: Despliegue automático con SSM en cada EC2

- El workflow llama a aws ssm send-command a la instancia EC2 correspondiente.

- Dentro del comando SSM se ejecuta, en la EC2:
  - aws ecr get-login-password | docker login … → login en ECR **dentro de la instancia**.
  - docker pull <imagen>:latest → trae la nueva versión.
  - docker stop + docker rm del contenedor viejo (si existe).
  - docker run -d … levantando el contenedor:
    - DB: expone 3306 y usa volumen dbdata.
    - Backend: expone 3001 y recibe variables DB_HOST, DB_USER, etc.
    - Frontend: expone 80 para atender a los usuarios.

PASO 5: Ejecutar la práctica en clase

- En cada archivo modificar algo simple:
  - init.sql (DB),
  - un texto en server.js (backend),
  - un título en index.html (frontend).
- Hacer git add . , git commit –m “texto” y git push a main.
- Observan en la pestaña **Actions** cómo se ejecuta el pipeline de la capa modificada:
  - Build → Push a ECR → Deploy vía SSM.

PASO 6: Validar el despliegue en AWS

- En las instancias EC2, comprobar con:
  - docker ps → ver contenedores tienda-db, tienda-backend, tienda-frontend corriendo.
  - docker logs para revisar errores si algo falla.
  - En la DB: SELECT * FROM productos; para validar los registros iniciales. (Comando: sudo docker exec -it tienda-db mysql -u root -padmin123 -e "USE tienda_perritos; SELECT * FROM productos;")
- Desde el navegador, abrir la IP pública del frontend y verificar que:
  - La app carga,
  - El backend responde,
  - Los productos de la BD se muestran correctamente.

Evidencia requerida

El estudiante debe entregar un **archivo PPT con evidencias breves**, que incluya como mínimo:

- Captura del **repositorio GitHub** y estructura del proyecto.
- Capturas de los **workflows en GitHub Actions** ejecutados correctamente.
- Evidencia de **secrets configurados** (sin mostrar valores sensibles).
- Capturas de **Amazon ECR** mostrando las imágenes publicadas y sus tags.
- Capturas de **logs del pipeline** (build, push y deploy).
- Evidencia de los **contenedores corriendo en EC2**.
- Captura de la **aplicación funcionando en el navegador**.
- Breve explicación del flujo **CI/CD completo**.